

Exemplo de Execução

Missão Prática #01 - Módulo 1: Planejamento e Escopo com IA

Este documento mostra como ficaria uma entrega completa da missão usando a transcrição da reunião de discovery do projeto RouteWise. Use como referência de formato e profundidade: não copie o conteúdo.

Parte 1 - Output completo do Requirements Copilot

Modelo: Gemini | Temperatura: 0.3 | System prompt: requirements-copilot-system-prompt.md v1.1
Input: transcricao-discovery-routewise.md (35 min, reunião RouteWise, 14/04/2025)

Este é o relatório de engenharia de requisitos para o projeto RouteWise, baseado na transcrição da reunião de discovery de 14/04/2025.

1. Mapa de Domínios

Domínio	Descrição	Confiança
Telemetria e Rastreamento	Captura e processamento de posição e velocidade em tempo real.	Alta
Gestão de Incidentes	Regras de alerta, monitoramento de excesso de velocidade e escalonamento.	Média
Gestão de Ativos (IoT)	Monitoramento de saúde, bateria e conectividade dos dispositivos GPS.	Alta
Analytics Operacional	Consolidação de dados para dashboards gerenciais e relatórios de RH.	Média
Segurança Comportamental	Deteccção de padrões de condução (frenagem brusca) via acelerômetro.	Baixa

2. Mapa de Stakeholders

Nome/Papel	Tipo	Requisitos que defendem	Conflitos / Observações
Carlos Mendonça (Dir. Operações)	Negócio	Redução de multas/acidentes, dashboards automáticos, regras flexíveis.	[CONFLITO] Deseja “tempo real” mas não define latência técnica.
Priya (TI/Infra)	Técnico	Viabilidade de hardware, integração de APIs, conformidade LGPD.	Preocupada com a inconsistência de hardware (antigo vs. novo).
RH (Não presente)	Usuário Final	Exportação de ocorrências para avaliação de desempenho.	[VALIDAR] Necessidade de integração via API ou exportação manual.
Operador de Despacho	Usuário Final	Mapa em tempo real e interface de alertas.	Não participou da reunião; requisitos inferidos por Carlos.

3. Estrutura de Épicos

Título	Descrição	Complexidade	Justificativa
EP01: Monitoramento de Velocidade Crítica	Processamento de telemetria e geração de alertas imediatos.	G	Exige processamento de stream de dados e motor de regras configurável.
EP02: Gestão de Saúde de Dispositivos	Dashboard de status (online/offline) e telemetria de bateria.	M	Depende da normalização de dados entre modelos de GPS distintos.
EP03: Inteligência de Dados e Reporting	Automação de relatórios semanais e mensais para diretoria e RH.	M	Complexidade reside na agregação de dados históricos de 140 veículos.
EP04: Telemetria Comportamental (v2)	Deteção de frenagem brusca via acelerômetro.	P	Lógica simples, mas depende da substituição de hardware antigo.

4. User Stories

US01: Alerta de Excesso de Velocidade

Card: Como Operador de Despacho, quero receber alertas automáticos de excesso de velocidade, para que eu possa intervir imediatamente e evitar multas ou acidentes.

Validação INVEST:

- Independent: PASS
- Negotiable: PASS
- Valuable: PASS
- Estimable: PASS
- Small: PASS (se focado apenas no alerta visual)
- Testable: PASS

CrITÉrios de Aceite (Gherkin):

Cenário: Veículo ultrapassa limite configurado

Dado que o veículo "ABC-1234" tem um limite de velocidade de 80km/h configurado para sua rota

Quando o dispositivo enviar uma coordenada com velocidade de 85km/h

Então o sistema deve gerar um alerta visual e sonoro no console do operador em até [A CONFIRMAR] segundos.

Cenário: Veículo retorna à velocidade permitida

Dado que um alerta de velocidade está ativo para o veículo "ABC-1234"

Quando o dispositivo enviar uma coordenada com velocidade de 75km/h

Então o sistema deve marcar o alerta como "Normalizado" no histórico.

Dependências: EP01, conectividade estável do dispositivo.

Notas Técnicas: O motor de regras deve considerar a latência da rede móvel.

US02: Escalonamento de Alerta Não Atendido

Card: Como Diretor de Operações, quero que alertas não visualizados pelo operador sejam escalonados para a coordenação, para que nenhuma infração crítica passe despercebida.

Validação INVEST: [INVEST-FAIL: Small]: a lógica de hierarquia de usuários e tempos de escalonamento precisa de detalhamento.

Critérios de Aceite (Gherkin):

Cenário: Alerta ignorado pelo operador

Dado que um alerta de velocidade foi gerado às 10:00:00

E o operador não realizou o "Acknowledge" (ciência) no sistema

Quando o tempo de [A CONFIRMAR] minutos for atingido

Então o sistema deve enviar uma notificação para o perfil "Coordenador de Rota".

Dependências: US01.

US03: Monitoramento de Conectividade (Heartbeat)

Card: Como Técnico de Dispositivos, quero visualizar quais rastreadores estão offline, para que eu possa identificar falhas de hardware ou áreas sem cobertura.

Validação INVEST: PASS.

Critérios de Aceite (Gherkin):

Cenário: Dispositivo para de enviar sinal

Dado que o dispositivo "GPS-99" envia sinais a cada 30 segundos

Quando o sistema não receber nenhum sinal por mais de [A CONFIRMAR] minutos

Então o status do dispositivo no dashboard deve mudar para "Offline" (cor vermelha).

Dependências: EP02.

5. Perguntas em Aberto

1. Qual a latência máxima aceitável para o "tempo real"? → Impacto: define a arquitetura de processamento (Batch vs. Stream) e custos de infraestrutura.
2. Qual o tempo exato (em minutos) para o escalonamento de um alerta do operador para o coordenador? → Impacto: define a lógica do motor de notificações.
3. O sistema de RH possui API para recebimento de dados ou aceita apenas importação de arquivos (CSV/Excel)? → Impacto: pode inviabilizar a automação total do EP03.
4. Quais são os limites de velocidade específicos por tipo de veículo e por rota? → Impacto: define a complexidade do banco de dados de regras.
5. Como o sistema deve se comportar em áreas de sombra (sem sinal)? Deve retroagir alertas quando o sinal voltar? → Impacto: integridade do histórico de multas.

6. Flags de Risco

- [ESPECIFICAÇÃO INVENTADA]: Defini o status "Normalizado" na US01 para fechamento de ciclo, mas o stakeholder não mencionou como o alerta termina.
- [DEPENDÊNCIA NÃO MAPEADA]: A US01 pressupõe que o sistema conhece o limite de velocidade da via/rota. Não foi mencionado se haverá integração com mapas (Google Maps/OSM) para limites dinâmicos.
- [VIABILIDADE TÉCNICA SILENCIOSA]: O uso de acelerômetro para "frenagem brusca" exige calibração específica por modelo de caminhão. Sem isso, o volume de falsos positivos será alto.
- [GOLD PLATING]: Adicionei "alerta sonoro" na US01 para garantir a atenção do operador, embora Carlos tenha mencionado apenas "olhar para a tela".

7. Cards Prontos para Jira

Épico: EP03: Inteligência de Dados e Reporting

Feature: Relatórios Automáticos

Título: Como Diretor de Operações, quero relatórios semanais automáticos

Tipo: Story

Story Points: 5: [envolve agregação de dados de múltiplas fontes e agendamento de disparo]

Labels: Analytics, Média

Para que: eu pare de gastar 2 horas semanais consolidando dados manualmente no Excel para o board.

Critérios de Aceite:

Cenário: Geração de relatório semanal de performance

Dado que é segunda-feira, 08:00 AM

Quando o sistema processar os dados da semana anterior

Então deve gerar um PDF contendo: total de alertas por tipo, veículos com mais ocorrências e ranking de motoristas.

E enviar para o e-mail cadastrado do Diretor de Operações.

Dependências: US01, US03

Definition of Ready: [⚠️ Pendente: confirmar campos exatos do relatório com Carlos]

8. Dependências Não Declaradas

- Provedor de Mapas (API) → Bloqueia US01 (regras por rota) → Necessário contratar Google Maps API ou similar para geofencing e limites de velocidade.
- Serviço de Notificação (SMTP/Push) → Bloqueia US02 (escalonamento) → Necessário configurar servidor de e-mail ou serviço de mensageria.
- Firmware Update → Bloqueia EP04 (acelerômetro) → Necessário validar se os dispositivos novos já transmitem dados de acelerômetro no protocolo atual.

9. Diagrama de Fluxo (Mermaid)

flowchart TD

```

A[Dispositivo GPS] -->|Envia Telemetria| B{Processador}
B -->|Velocidade > Limite| C[Gera Alerta Visual/Sonoro]
B -->|Sem sinal > X min| D[Gera Alerta Offline]
C --> E{Operador Atende?}
E -->|Sim| F[Log de Ciência]
E -->|Não [?]| G[Escalona para Coordenador]
F --> H[Dashboard Gerencial]
G --> H
D --> H
H --> I[Relatório Semanal/Mensal]
I --> J[Fim do Fluxo]

```

⚠️ *Este output é um rascunho analítico. Requer revisão humana antes de entrar em sprint. Valide: viabilidade técnica, compliance/LGPD e dependências não mapeadas.*

Parte 2 - Dois pontos de falha identificados

Falha 1 - Especificação inventada (US01)

O que o modelo gerou: critério de aceite com “[A CONFIRMAR] segundos” para latência do alerta.

O que está errado: o modelo não inventou um número, isso é comportamento correto do protocolo de ambiguidade. Mas ao apresentar para o Carlos sem contexto, o [A CONFIRMAR] vai aparecer como lacuna sem dono. Ninguém se responsabilizou por obter esse dado antes do sprint planning.

Tipo de falha: [ESPECIFICAÇÃO INVENTADA] (ausência de SLA com stakeholder responsável identificado)

→ Pergunta para o Carlos: “Qual o atraso máximo aceitável entre o GPS registrar a velocidade e o alerta aparecer no painel? Acima de quantos segundos você consideraria uma falha operacional?”

Falha 2 - Dependência não mapeada (US01 e US02)

O que o modelo gerou: US01 pressupõe uma base de dados de limites de velocidade por via. Essa base não foi mencionada em nenhum momento na transcrição.

O que está errado: a história entra no backlog como se a base existisse. Na Sprint 1 alguém vai perguntar de onde vem o limite de 80km/h por rodovia, e ninguém vai saber responder.

Tipo de falha: [DEPENDÊNCIA NÃO MAPEADA]: Provedor de Mapas com Speed Limits API (GIS) não confirmado.

→ Ação: bloquear US01 na Definition of Ready até contratar e confirmar o provedor de mapas.

Parte 3 - Ajuste no prompt e comparação V1 vs V2

O que mudei no prompt

Adicionei ao bloco do Protocolo de Ambiguidade do system prompt a seguinte instrução:

“Para cada dependência técnica externa identificada (APIs, integrações, provedores), gere uma linha na seção DEPENDÊNCIAS NÃO DECLARADAS indicando: (1) o nome do sistema externo, (2) qual User Story bloqueia, e (3) a pergunta de confirmação para o stakeholder técnico.”

Por quê: a V1 identificou a dependência da API de mapas no bloco de FLAGS DE RISCO, mas de forma genérica. Queria que o modelo fosse mais preciso, nomeando o sistema externo, a história bloqueada e gerando a pergunta de confirmação automaticamente.

Comparação V1 vs V2

V1 - antes do ajuste	V2 - depois do ajuste
[DEPENDÊNCIA NÃO MAPEADA]: Provedor de Mapas (Speed Limits API) → Bloqueia US01 → Contratar e integrar API de limites de velocidade por geolocalização.	DEPENDÊNCIAS NÃO DECLARADAS → Sistema externo: Provedor de Mapas / Speed Limits API. Bloqueia: US01. Pergunta para o arquiteto: “Temos contrato com Google Maps Platform ou HERE Maps? Precisamos do endpoint de speed limits por coordenada antes da Sprint 1.”

O que melhorou: a V2 gerou a pergunta de confirmação automaticamente para cada dependência, com o sistema externo nomeado e a história bloqueada identificada. Isso eliminou uma rodada de interpretação manual do output.

O que ainda persiste: o [A CONFIRMAR] de latência na US01 continua sem dono atribuído: o modelo não sabe quem no time é responsável por obter esse dado do Carlos. Isso seria o ajuste da V3.

Este exemplo representa nível intermediário da rubrica.

Para nível avançado: use uma transcrição do seu próprio projeto, envie as User Stories geradas para o seu PO real, documente as correções recebidas e entregue a V3 com as revisões aplicadas.

Reflexão

O valor do Requirements Copilot não está nas User Stories que ele gera: está nos flags que ele sinaliza antes do primeiro sprint. Uma dependência não mapeada ou uma especificação inventada identificada aqui custa horas. A mesma lacuna descoberta na Sprint 3 custa semanas. A iteração de prompt documentada neste exemplo não é um exercício acadêmico: é a prática de tornar visível o que o modelo não sabe antes que vire blocker.